

Лабораторная работа №5

Построение графиков

Доберштейн А. С.

Российский университет дружбы народов, Москва, Россия

Информация

- Доберштейн Алина Сергеевна
- НФИбд-02-22
- Российский университет дружбы народов

Цель работы

Основная цель работы — освоить синтаксис языка Julia для построения графиков.

Основные пакеты для работы с графиками в Julia

[62]:

```
using Pkg
Pkg.add("Plots")
Pkg.add("PyPlot")
Pkg.add("Plotly")
Pkg.add("UnicodePlots")
Pkg.build("Conda")
```

```
Resolving package versions...
No Changes to `C:\Users\MI\.julia\environments\v1.11\Project.toml`
No Changes to `C:\Users\MI\.julia\environments\v1.11\Manifest.toml`
Resolving package versions...
No Changes to `C:\Users\MI\.julia\environments\v1.11\Project.toml`
No Changes to `C:\Users\MI\.julia\environments\v1.11\Manifest.toml`
Resolving package versions...
No Changes to `C:\Users\MI\.julia\environments\v1.11\Project.toml`
No Changes to `C:\Users\MI\.julia\environments\v1.11\Manifest.toml`
Resolving package versions...
No Changes to `C:\Users\MI\.julia\environments\v1.11\Project.toml`
No Changes to `C:\Users\MI\.julia\environments\v1.11\Manifest.toml`
Building Conda → `C:\Users\MI\.julia\scratchspaces\44cfe95a-1eb2-52ea-b672-e2afdf69b78f\8f06b0cfa4c514c7b9546756dbae91fcfbc92dc9\build.log`
```

[63]:

```
using Plots
```

Рис. 1: Основные пакеты для работы с графиками в Julia

Основные пакеты для работы с графиками в Julia

```
[64]:  
# задание функции:  
f(x) = (3x.^2 + 6x .- 9).*exp.(-0.3x)  
# генерирование массива значений x в диапазоне от -5 до 10 с шагом 0,1  
# (шаг задан через указание длины массива):  
x = collect(range(-5,10,length=151))  
# генерирование массива значений y:  
y = f(x);  
  
[65]:  
# указывается, что для построения графика используется gr():  
gr()  
  
Plots.GRBackend()
```

Рис. 2: Основные пакеты для работы с графиками в Julia

Основные пакеты для работы с графиками в Julia

[66]:

```
# задание опций при построении графика  
# (название кривой, подписи по осям, цвет графика):  
plot(x,y,  
      title="A simple curve",  
      xlabel="Variable x",  
      ylabel="Variable y",  
      color="blue")
```

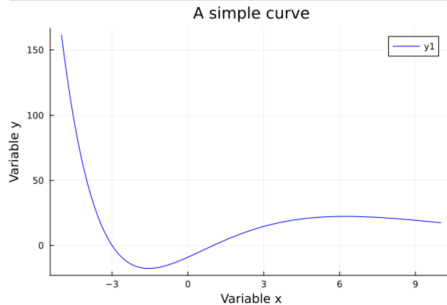


Рис. 3: Основные пакеты для работы с графиками в Julia

Основные пакеты для работы с графиками в Julia

```
# указывается, что для построения графика используется pyplot():  
pyplot()
```

```
Plots.PyPlotBackend()
```

```
[68]:
```

```
# задание опций при построении графика  
# (название кривой, подписи по осям, цвет графика):  
plot(x,y,  
      title="Простая кривая",  
      xlabel="Переменная x",  
      ylabel="Переменная y",  
      color="blue")
```

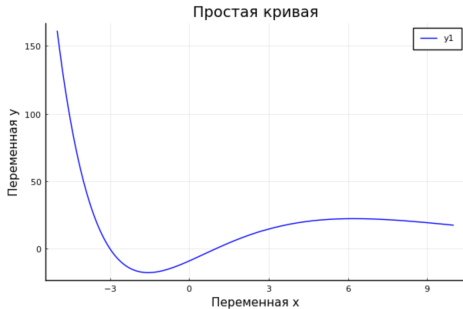


Рис. 4: Основные пакеты для работы с графиками в Julia

Основные пакеты для работы с графиками в Julia

```
unicodeplots()
Plots.UnicodePlotsBackend()

[72]:
# задание опций при построении графика
# (название кривой, подписи по осям, цвет графика):
plot(x,y,
      title="A simple curve",
      xlabel="Variable x",
      ylabel="Variable y",
      color="blue")
```

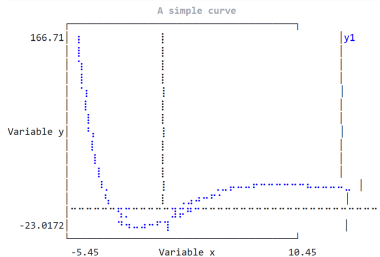


Рис. 5: Основные пакеты для работы с графиками в Julia

Опции при построении графика

```
# указывается, что для построения графика используется pyplot():  
pyplot()
```

```
Plots.PyPlotBackend()
```

```
[75]:
```

```
# задание функции sin(x):  
sin_theor(x) = sin(x)  
# построение графика функции sin(x):  
plot(sin_theor, title="sin(x)",  
     xlabel="Переменная x",  
     ylabel="Переменная y")
```

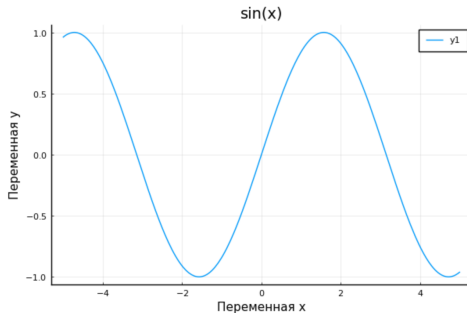


Рис. 6: Опции при построении графика

Опции при построении графика

```
# задание функции разложения исходной функции в ряд Тейлора:  
sin_taylor(x) = [(-1)**i*x**(2*i+1)/factorial(2*i+1) for i in 0:4] |> sum  
# построение графика функции sin_taylor(x):  
plot(sin_taylor, title="sin_taylor",  
      xlabel="Переменная x",  
      ylabel="Переменная y")
```

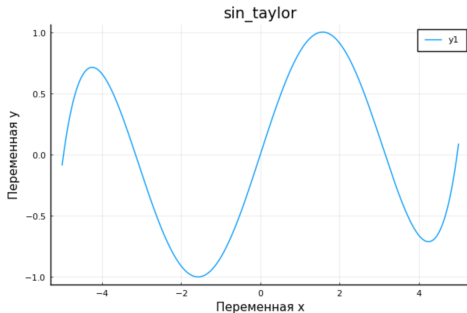


Рис. 7: Опции при построении графика

Опции при построении графика

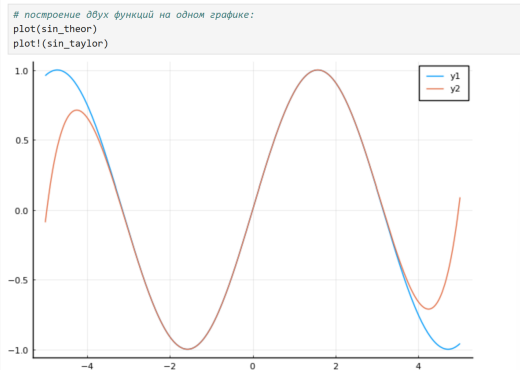


Рис. 8: Опции при построении графика

Опции при построении графика

```
plot(
# функция sin(x):
sin_taylor,
# подпись в легенде, цвет и тип линии:
label = "sin(x), разложение в ряд Тейлора",
line=(:blue, 0.3, 6, :solid),
# размер графика:
size=(800, 500),
# параметры отображения значений по осям
xticks = (-5:0.5:5),
yticks = (-1:0.1:1),
xtickfont = font(12, "Times New Roman"),
ytickfont = font(12, "Times New Roman"),
# подписи по осям:
ylabel = "y",
xlabel = "x",
# название графика:
title = "Разложение в ряд Тейлора",
# поворот значений, заданный по оси x:
xrotation = rad2deg(pi/4),
# заливка области графика цветом:
fillrange = 0,
fillalpha = 0.5,
fillcolor = :lightgoldenrod,
# задание цвета фона:
background_color = :ivory
)
plot!(
# функция sin_theor:
sin_theor,
# подпись в легенде, цвет и тип линии:
label = "sin(x), теоретическое значение",
line=(:black, 1.0, 2, :dash))
```

Рис. 9: Опции при построении графика

Опции при построении графика

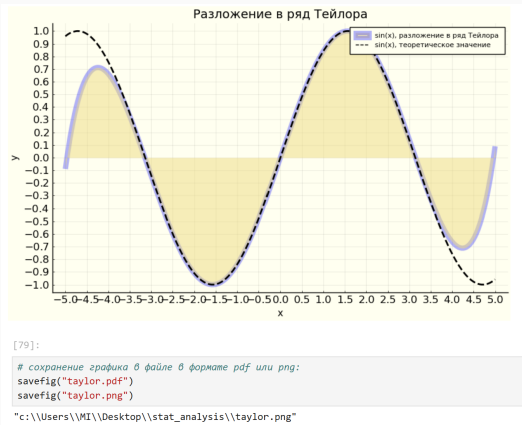


Рис. 10: Опции при построении графика

Точечный график

```
# параметры распределения точек на плоскости:  
x = range(1,10,length=10)  
y = rand(10)  
# параметры построения графика:  
plot(x, y,  
      seriestype = :scatter,  
      title = "Точечный график"  
)
```

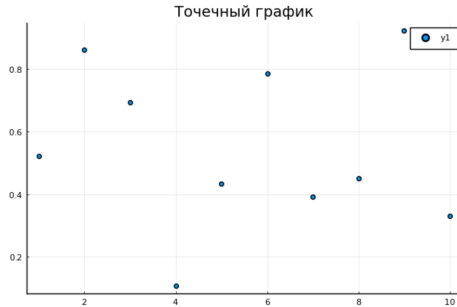


Рис. 11: Точечный график

Точечный график

```
# параметры распределения точек на плоскости:  
n = 50  
x = rand(n)  
y = rand(n)  
ms = rand(50) * 30  
# параметры построения графика:  
scatter(x, y, markersize=ms)
```

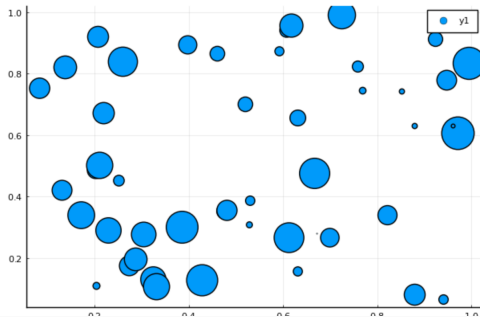


Рис. 12: Точечный график

Точечный график

```
# параметры распределения точек в пространстве:  
n = 50  
x = rand(n)  
y = rand(n)  
z = rand(n)  
ms = rand(50) * 30  
# параметры построения графика:  
scatter(x, y, z, markersize=ms, xlabel="x", ylabel="y", zlabel="z", title="3-мерный точечный график")
```

3-мерный точечный график

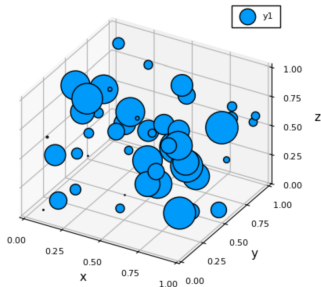


Рис. 13: Точечный график

```
# массив данных от 0 до 10 с шагом 0.01:  
x = collect(0:0.01:9.99)  
# экспоненциальная функция со случайным сдвигом значений:  
y = exp.(ones(1000)*x) + 4000*randn(1000)  
# построение графика:  
scatter(x,y,markersize=3,alpha=.8,legend=false)
```

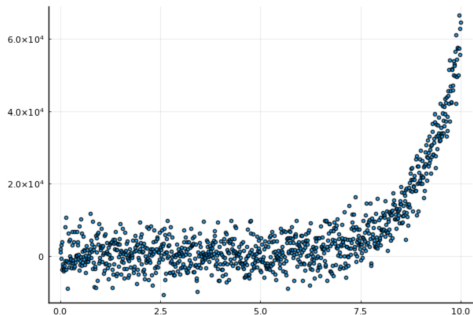


Рис. 14: Аппроксимация данных

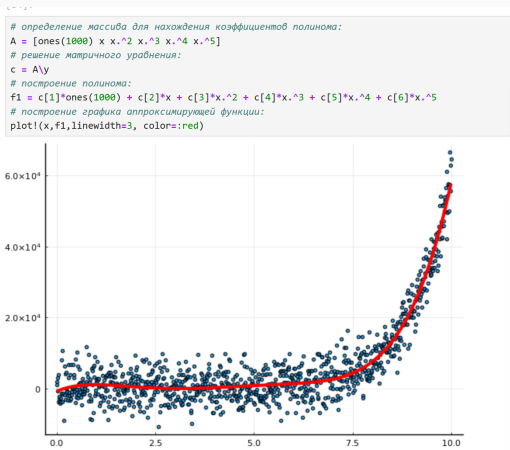


Рис. 15: Аппроксимация данных

Две оси ординат

[92]:

```
# пример случайной траектории
# (заданы обозначение траектории, легенда сверху справа, без сетки)
using Plots
gr()

p1 = plot(1:100, randn(100),
  ylabel="y1",
  leg=:topright,
  grid = :on,
)
```

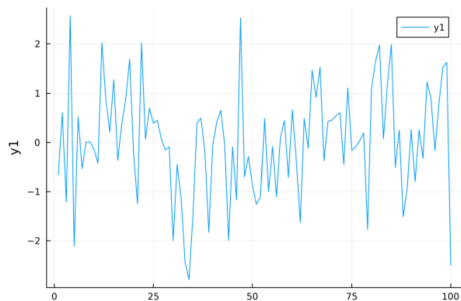


Рис. 16: Две оси ординат

Две оси ординат

```
# пример добавления на график второй случайной траектории
# (задано обозначение траектории и её цвет, легенда снизу справа, без сетки)
# задана рамка графика
p2 = twinx(p1)
plot!(p2, 1:100, randn(100)*10,
      c=:red,
      ylabel="y2",
      #axis=:right,
      leg=:bottomright,
      grid = :on,
      box = :on,
      #size=(600, 400)
)
```

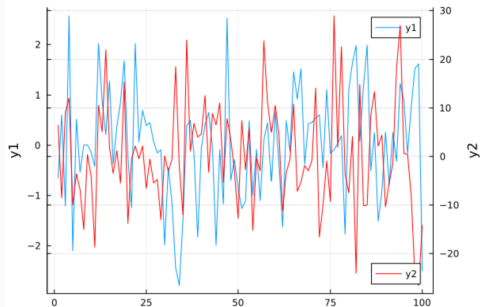


Рис. 17: Две оси ординат

Полярные координаты

```
# функция в полярных координатах:  
r(θ) = 1 + cos(θ) * sin(θ)^2  
# полярная система координат:  
θ = range(0, stop=2π, length=50)  
# график функции, заданной в полярных координатах:  
plot(θ, r.(θ),  
proj=:polar,  
lims=(0,1.5)  
)
```

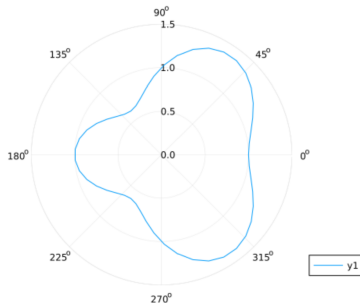


Рис. 18: Полярные координаты

Параметрический график

```
# параметрическое уравнение:  
x1(t) = sin(t)  
y1(t) = sin(2t)  
# построение графика:  
plot(x1, y1, 0, 2π, leg=false, fill=(0,:orange))
```

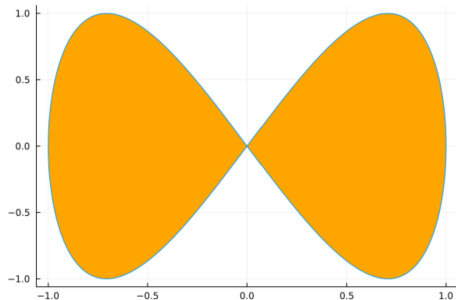


Рис. 19: Параметрический график

Параметрический график

```
# параметрическое уравнение  
t = range(0, stop=10, length=1000)  
x = cos.(t)  
y = sin.(t)  
z = sin.(5t)  
# построение графика:  
plot(x, y, z)
```

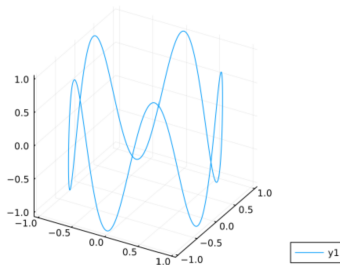


Рис. 20: Параметрический график

[97]:

```
f(x,y) = x^2 + y^2  
x = -10:10  
y = x  
surface(x, y, f)
```

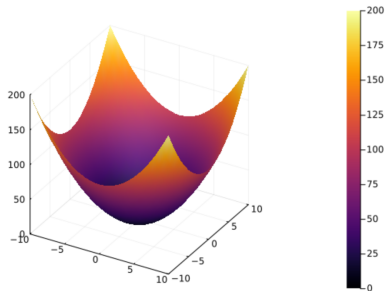


Рис. 21: График поверхности

График поверхности

```
# построение графика поверхности:
```

```
f(x,y) = x^2 + y^2
```

```
x = -10:10
```

```
y = x
```

```
plot(x, y, f,
```

```
linetype=:wireframe
```

```
)
```

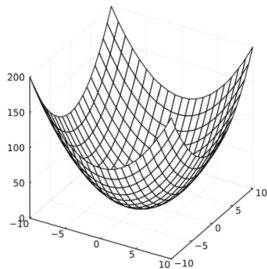


Рис. 22: График поверхности

График поверхности

[99]:

```
f(x,y) = x^2 + y^2  
x = -10:0.1:10  
y = x  
plot(x, y, f,  
linetype = :surface  
)
```

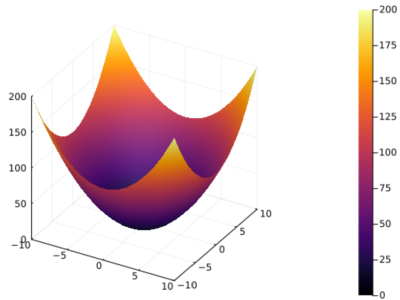


Рис. 23: График поверхности

График поверхности

[100]:

```
x=range(-2,stop=2,length=100)  
y=range(sqrt(2),stop=2,length=100)  
f(x,y) = x*y-x-y+1  
plot(x,y,f,  
      linetype=:surface,  
      c=cgrad([:red,:blue]),  
      camera=(-30,30),  
      )
```

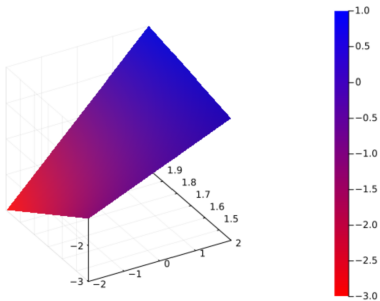


Рис. 24: График поверхности

[101]:

```
x = 1:0.5:20  
y = 1:0.5:10  
g(x, y) = (3x + y ^ 2) * abs(sin(x) + cos(y))  
plot(x,y,g,  
      linestyle = :surface,  
      )
```

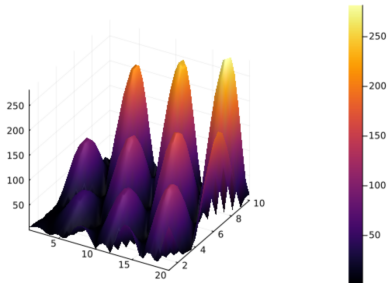


Рис. 25: Линии уровня

```
[102]:
```

```
contour(x, y, g)
```

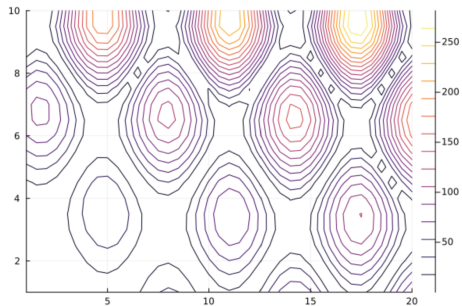


Рис. 26: Линии уровня

```
[103]:
```

```
p = contour(x, y, g,  
            fill=True)  
plot(p)
```

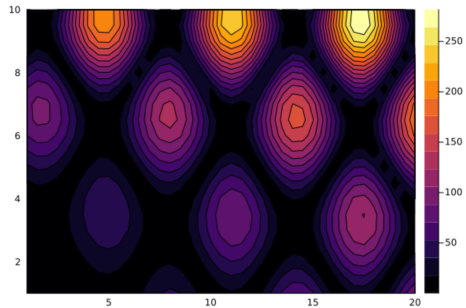


Рис. 27: Линии уровня

[104]:

```
# определение переменных:  
X = range(-2, stop=2, length=100)  
Y = range(-2, stop=2, length=100)  
# определение функции:  
h(x, y) = x^3 - 3x + y^2  
# построение поверхности:  
plot(X,Y,h,  
linetype = :surface  
)
```

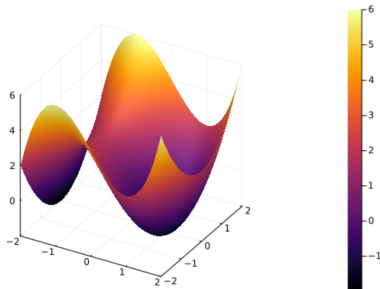


Рис. 28: Векторные поля

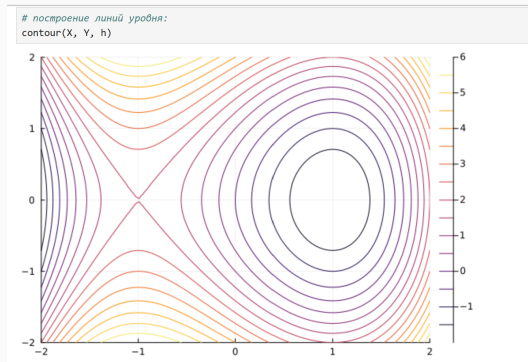


Рис. 29: Векторные поля

[106]:

```
# градиент:  
x = range(-2, stop=2, length=12)  
y = range(-2, stop=2, length=12)  
# производная от исходной функции:  
dh(x, y) = [3x^2 - 3; 2y] / 25  
# построение векторного поля:  
quiver!(x, y, quiver=dh, c=:blue)  
# коррекция области видимости графика:  
xlims!(-2, 2)  
ylims!(-2, 2)
```

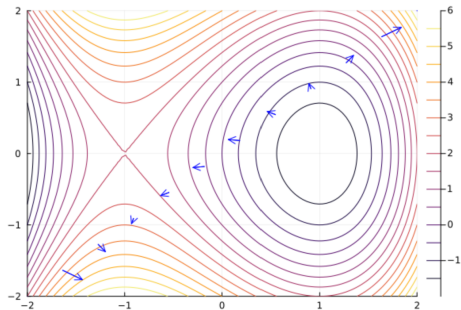


Рис. 30: Векторные поля

[107]:

```
pyplot()  
# построение поверхности:  
i = 0  
X = Y = range(-5, stop=5, length=40)  
surface(X, Y, (x,y) -> sin(x+10sin(i))+cos(y))
```

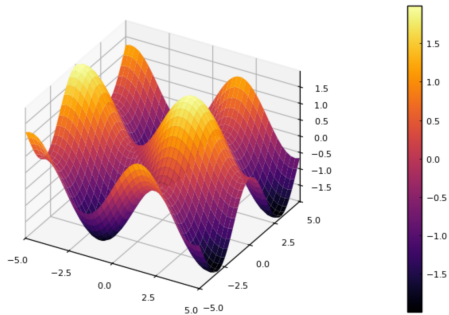


Рис. 31: Анимация

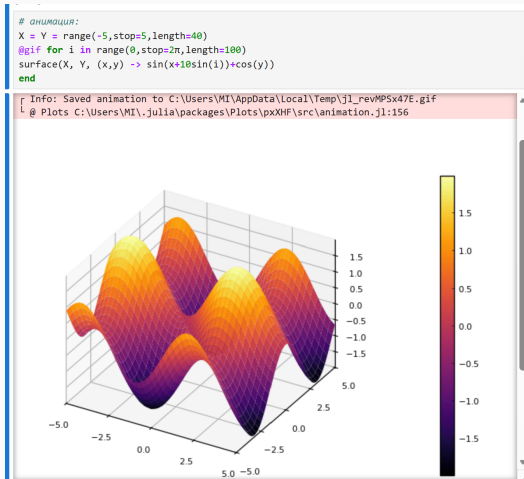


Рис. 32: Анимация


```
# радиус малой окружности:
r1 = 1
# коэффициент для построения большой окружности:
k = 3
# число отсчётов:
n = 100

# массив значений угла θ:
# theta from 0 to 2pi ( + a little extra)
θ = collect(0:2*π/100:2*π+2*π/100)
# массивы значений координат:
X = r1*k*cos.(θ)
Y = r1*k*sin.(θ)

# задаём оси координат:
plt=plot(5,xlim=(-4,4),ylim=(-4,4), c=:red, aspect_ratio=1,legend=false, framestyle=:origin)
# большая окружность:
plot!(plt, X,Y, c=:blue, legend=false)
i = 50
t = θ[1:i]
# гипоциклоида:
x = r1*(k-1)*cos.(t) + r1*cos.((k-1)*t)
y = r1*(k-1)*sin.(t) - r1*sin.((k-1)*t)
plot!(x,y, c=:red)
# малая окружность:
xc = r1*(k-1)*cos(t[end]) .+ r1*cos.(θ)
yc = r1*(k-1)*sin(t[end]) .+ r1*sin.(θ)
plot!(xc,yc,c=:black)
# радиус малой окружности:
x1 = transpose([r1*(k-1)*cos(t[end]) x[end]])
y1 = transpose([r1*(k-1)*sin(t[end]) y[end]])
```

Рис. 33: Анимация

```
plot!(x1,y1,markershape=:circle,markersize=4,c=:black)
scatter!([x[end]], [y[end]], c=:red, markerstrokecolor=:red)
anim = @animate for i in 1:n
# задаём оси координат:
    plt=plot(5,xlim=(-4,4),ylim=(-4,4), c=:red, aspect_ratio=1,legend=false, framestyle=:or
# большая окружность:
    plot!(plt, X,Y, c=:blue, legend=false)
    t = 0[1:i]
# гипотенуза:
    x = r1*(k-1)*cos.(t) + r1*cos.((k-1)*t)
    y = r1*(k-1)*sin.(t) - r1*sin.((k-1)*t)
    plot!(x,y, c=:red)
# малая окружность:
    xc = r1*(k-1)*cos(t[end]) .+ r1*cos.(θ)
    yc = r1*(k-1)*sin(t[end]) .+ r1*sin.(θ)
    plot!(xc,yc,c=:black)
# радиус малой окружности:
    x1 = transpose([r1*(k-1)*cos(t[end]) x[end]])
    y1 = transpose([r1*(k-1)*sin(t[end]) y[end]])
    plot!(x1,y1,markershape=:circle,markersize=4,c=:black)
    scatter!([x[end]], [y[end]], c=:red, markerstrokecolor=:red)
end
```

```
will be ignored
sys:1: UserWarning: No data for colormapping provided via 'c'. Parameters 'vmin', 'vmax'
will be ignored
sys:1: UserWarning: No data for colormapping provided via 'c'. Parameters 'vmin', 'vmax'
will be ignored
sys:1: UserWarning: No data for colormapping provided via 'c'. Parameters 'vmin', 'vmax'
will be ignored
sys:1: UserWarning: No data for colormapping provided via 'c'. Parameters 'vmin', 'vmax'
will be ignored
sys:1: UserWarning: No data for colormapping provided via 'c'. Parameters 'vmin', 'vmax'
will be ignored
```

```
Animation("C:\\Users\\MI\\AppData\\Local\\Temp\\jl_WnBQHw", ["000001.png", "000002.png",
"000003.png", "000004.png", "000005.png", "000006.png", "000007.png", "000008.png", "0000
```

```
[110]:
```

```
gif(anim, "hypocycloid.gif")
```

```
[ Info: Saved animation to c:\Users\MI\Desktop\stat_analysis\hypocycloid.gif  
@ Plots C:\Users\MI\julia\packages\Plots\pxXHF\src\animation.jl:156
```

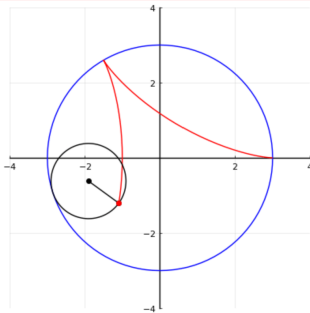


Рис. 35: Анимация

```
# подключение пакета Statistics:  
import Pkg  
Pkg.add("Statistics")  
using Statistics
```

Рис. 36: Errorbars

Errorbars

```
sds = [1, 1/2, 1/4, 1/8, 1/16, 1/32]
n = 10
y = [mean(sd*randn(n)) for sd in sds]
errs = 1.96 * sds / sqrt(n)
plot(y,
      ylims = (-1,1),
)
```

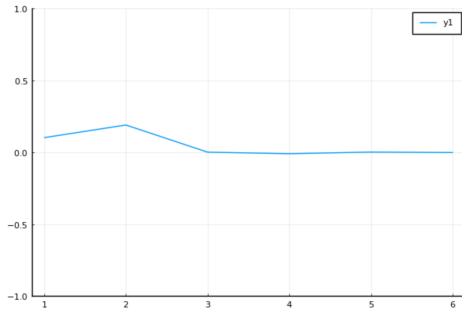


Рис. 37: Errorbars

Errorbars

```
plot(y,  
     ylims = (-1,1),  
     err = errs  
)
```

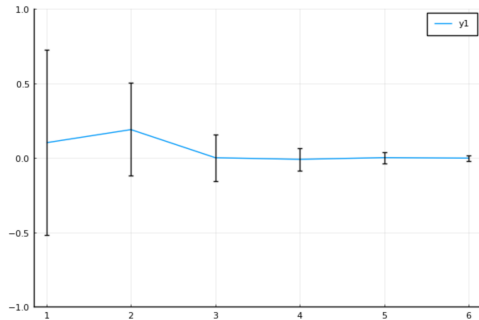


Рис. 38: Errorbars

```
[115]:
```

```
plot(y,  
     ribbon=errs,  
     fill=:cyan  
)
```

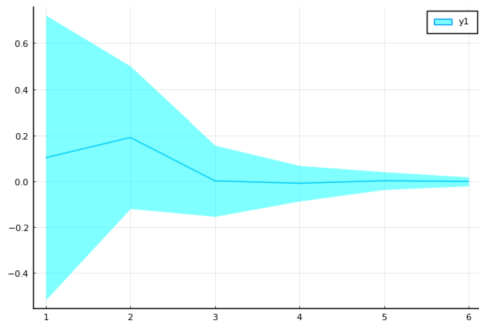
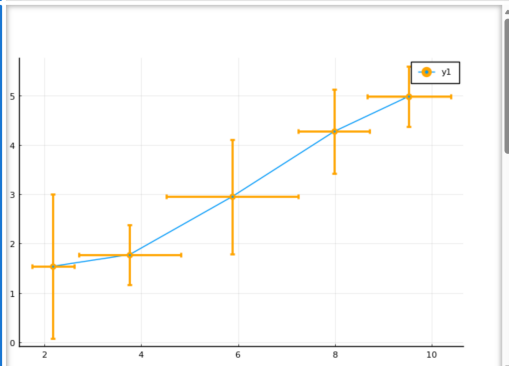


Рис. 39: Errorbars

Errorbars

```
n = 10
x = [(rand()+1) .* randn(n) .+ 2i for i in 1:5]
y = [(rand()+1) .* randn(n) .+ i for i in 1:5]
f(v) = 1.96std(v) / sqrt(n)
xerr = map(f, x)
yerr = map(f, y)
x = map(mean, x)
y = map(mean, y)
plot(x, y,
     xerr = xerr,
     yerr = yerr,
     marker = stroke(2, :orange))
```



Errorbars

```
plot(x, y,  
     xerr = (0.5xerr, 2xerr),  
     yerr = (0.5yerr, 2yerr),  
     marker = stroke(2, :orange)  
)
```

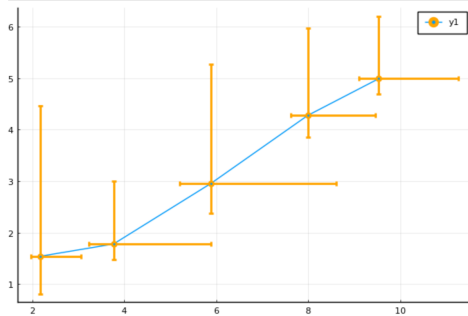


Рис. 41: Errorbars

Использование пакета Distributions

```
import Pkg
Pkg.add("Distributions")
using Distributions
```

```
Resolving package versions...
Updating `C:\Users\MI\.julia\environments\v1.11\Project.toml`
[31c24e10] + Distributions v0.25.122
No Changes to `C:\Users\MI\.julia\environments\v1.11\Manifest.toml`
```

```
[119]:
```

```
pyplot()
ages = rand(15:55, 1000)
histogram(ages)
```

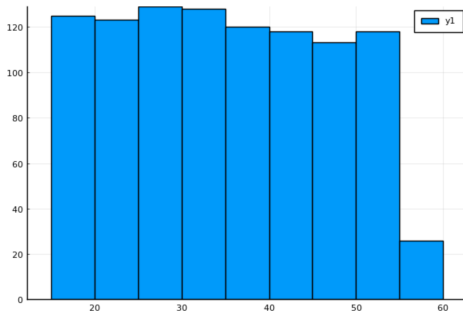


Рис. 42: Использование пакета Distributions

Использование пакета Distributions

```
d=Normal(35.0,10.0)
ages = rand(d,1000)
histogram(
  ages,
  label="Распределение по возрастам (года)",
  xlabel = "Возраст (лет)",
  ylabel= "Количество"
)
```

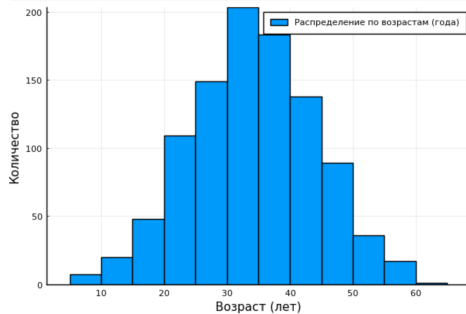


Рис. 43: Использование пакета Distributions

```
# подгружаем pyplot():  
pyplot()  
# построение серии графиков:  
x=range(-2,2,length=10)  
y = rand(10,4)  
plot(x,y,  
      layout=(4,1)  
)
```

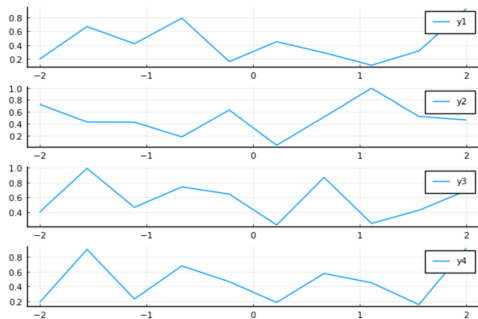


Рис. 44: Подграфики

```
plot(x,y,  
      layout=4  
)
```

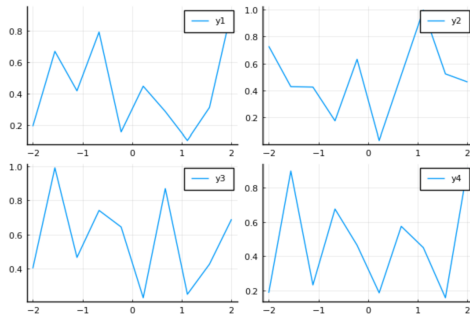


Рис. 45: Подграфики

```
plot(x,y,  
size=(600,300),  
layout = grid(4,1,heights=[0.2,0.3,0.4,0.1])  
)
```

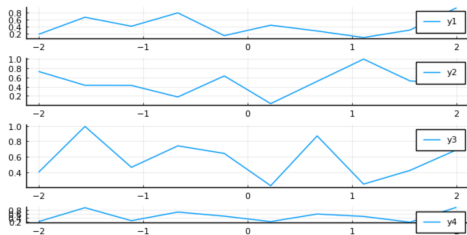
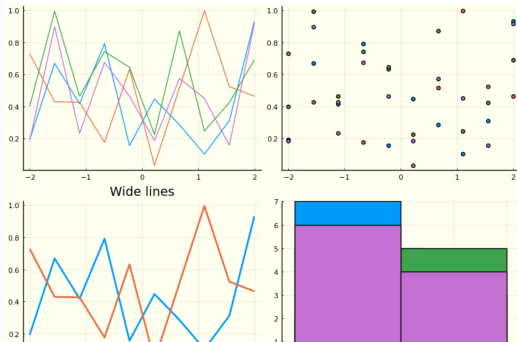


Рис. 46: Подграфики

Подграфики

```
# график в виде линий:  
p1 = plot(x,y)  
# график в виде точек:  
p2 = scatter(x,y)  
# график в виде линий с оформлением:  
p3 = plot(x,y[:,1:2],xlabel="Labelled plot of two columns",lw=2,title="Wide lines")  
# 4 гистограммы:  
p4 = histogram(x,y)  
plot(  
    p1,p2,p3,p4,  
    layout=(2,2),  
    legend=False,  
    size=(800,600),  
    background_color = :ivory  
)
```



```
seriestypes = [:step, :sticks, :bar, :hline, :vline, :path]
titles = ["step" "sticks" "bar" "hline" "vline" "path"]
plot(rand(20,1), st = seriestypes,
      layout = (2,3),
      ticks=nothing,
      legend=false,
      title=titles,
      m=3
      )
```

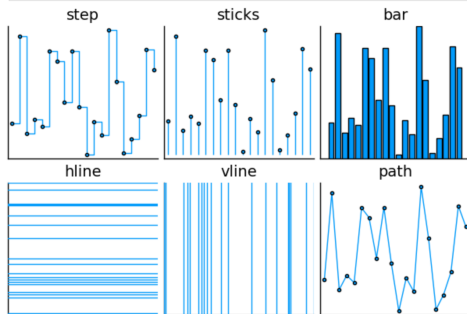


Рис. 48: Подграфики

[131]:

```
l = @layout [ a{0.3w} [grid(3,3)  
               b{0.2h} ]]  
  
plot(  
  rand(10,11),  
  layout = l, legend = false, seriestype = [:bar :scatter :path],  
  title = ["($i)" for j = 1:1, i=1:11], titleloc = :right, titlefont = font(8)  
)
```

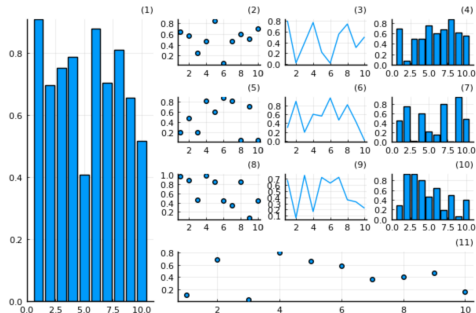


Рис. 49: Подграфики

Задания для самостоятельного выполнения

1. Постройте все возможные типы графиков (простые, точечные, гистограммы и т.д.) функции $y = \sin(x)$, $x = \overline{0, 2\pi}$. Отобразите все графики в одном графическом окне.

[140]:

```
pyplot()
```

```
Plots.PyPlotBackend()
```

[141]:

```
#1
y3(x) = sin(x)
x = range(0, stop=2π, length=100)
# график в виде линий:
p1 = plot(x,y3.(x))
# график в виде точек:
p2 = scatter(x,y3.(x))
# график в виде линий с оформлением:
p3 = plot(x,y3.(x),xlabel="Labelled plot of two columns",lw=2,title="Wide lines")
# 4 гистограммы:
p4 = histogram(x,y3.(x))
plot(
p1,p2,p3,p4,
layout=(2,2),
legend=false,
size=(800,600)
)
```

Рис. 50: Задание для самостоятельного выполнения №1

Задания для самостоятельного выполнения

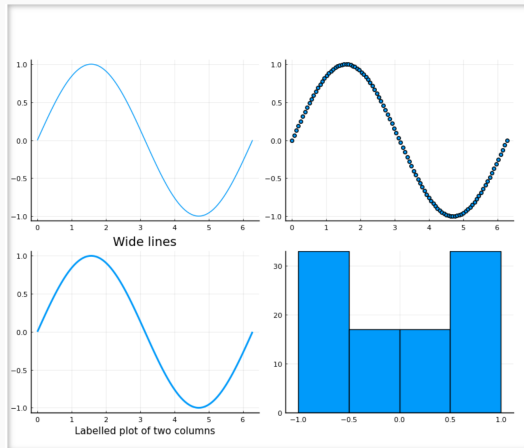


Рис. 51: Задание для самостоятельного выполнения №1

Задания для самостоятельного выполнения

2. Постройте графики функции $y = \sin(x)$, $x = \overline{0, 2\pi}$ со всеми возможными (сколько сможете вспомнить) типами оформления линий графика. Отобразите все графики в одном графическом окне.

```
[164]:
```

```
pyplot()
```

```
Plots.PyPlotBackend()
```

```
[165]:
```

```
#2
```

```
seriestypes = [:line, :scatter, :step, :sticks, :bar, :hline, :vline, :path]  
titles = ["line" "scatter" "step" "sticks" "bar" "hline" "vline" "path"]
```

```
plot(x, y3.(x), st = seriestypes,  
layout = 8,  
ticks=nothing,  
legend=false,  
title=titles,  
m=3  
)
```

Рис. 52: Задание для самостоятельного выполнения №2

Задания для самостоятельного выполнения

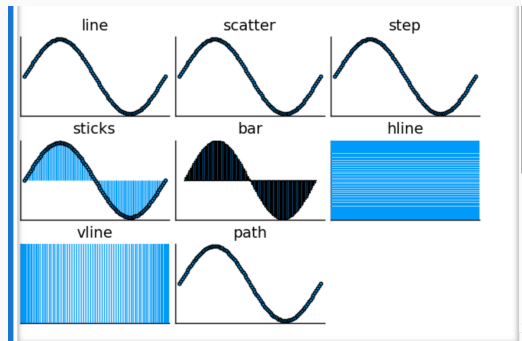


Рис. 53: Задание для самостоятельного выполнения №2

Задания для самостоятельного выполнения

3. Постройте график функции $y(x) = \pi x^2 \ln(x)$, назовите оси соответственно. Пусть цвет рамки будет зелёным, а цвет самого графика — красным. Задайте расстояние между надписями и осями так, чтобы надписи полностью умещались в графическом окне. Задайте шрифт надписей. Задайте частоту отметок на осях координат.

```
[167]:  
  
#3  
pyplot()  
y4(x)=pi*x^2*log(x)  
x = range(0,10,length=15)  
plot(x, y4.(x),  
      label = "y(x)", # подпись в легенде, цвет и тип линии:  
      c=:red,  
      xticks = (0:1:10), # параметры отображения значений по осям  
      yticks = (0:50:700),  
      xtickfont = font(10, "Times New Roman"),  
      ytickfont = font(10, "Times New Roman"),  
      ylabel = "y", # подписи по осям:  
      xlabel = "x",  
      title = "y(x)=pi*x^2*ln(x)", # название графика:  
      bordercolor = :green) # задание цвета рамки
```

Рис. 54: Задание для самостоятельного выполнения №3

Задания для самостоятельного выполнения

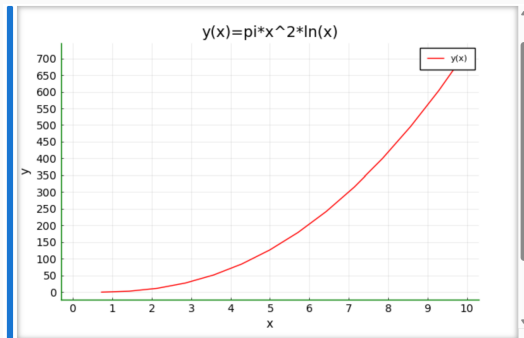


Рис. 55: Задание для самостоятельного выполнения №3

Задания для самостоятельного выполнения

4. Задайте вектор $x = (-2, -1, 0, 1, 2)$. В одном графическом окне (в 4-х подокнах) изобразите графически по точкам x значения функции $y(x) = x^3 - 3x$ в виде:
- точек,
 - линий,
 - линий и точек,
 - кривой.

Сохраните полученные изображения в файле `figure_familiya.png`, где вместо `familiya` укажите вашу фамилию.

```
[168]:  
  
#4  
pyplot()  
x = [-2, -1, 0, 1, 2]  
y5(x) = x^3 - 3*x  
  
#точки  
p1 = plot(x,y5.(x), line=:dot, title="точки")  
#линии  
p2 = plot(x,y5.(x), line=:dash, title="линии")  
#линии и точки  
p3 = plot(x,y5.(x), line=:dashdot, title="линии и точки")  
#кривая  
p4 = plot(x,y5.(x), line=:solid, title="кривая")  
  
plot(p1,p2,p3,p4)
```

Рис. 56: Задание для самостоятельного выполнения №4

Задания для самостоятельного выполнения

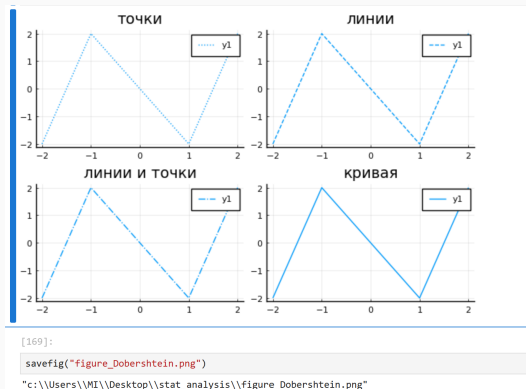


Рис. 57: Задание для самостоятельного выполнения №4

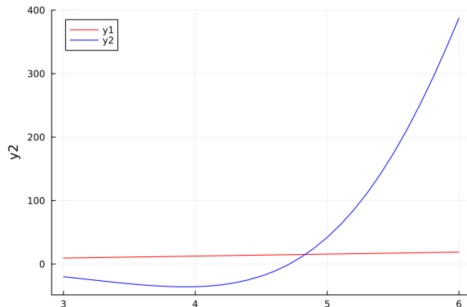
Задания для самостоятельного выполнения

5. Задайте вектор $x = (3, 3.1, 3.2, \dots, 6)$. Постройте графики функций $y_1(x) = \pi x$ и $y_2(x) = \exp(x) \cos(x)$ в указанном диапазоне значений аргумента x следующим образом:

- постройте оба графика разного цвета на одном рисунке, добавьте легенду и сетку для каждого графика; укажите недостатки у данного построения;
- постройте аналогичный график с двумя осями ординат.

[170]:

```
#5
gr()
x=collect(3:0.1:6)
y1(x) = pi*x
y2(x)=exp(x)*cos(x)
plot(x, y1.(x), c=:red, ylabel="y1", grid=:on)
plot!(x, y2.(x),c=:blue, ylabel="y2", grid=:on)
```



Задания для самостоятельного выполнения

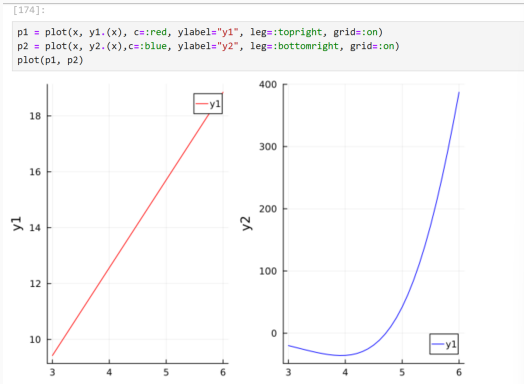


Рис. 59: Задание для самостоятельного выполнения №5

Задания для самостоятельного выполнения

6. Постройте график некоторых экспериментальных данных (придумайте сами), учитывая ошибку измерения.

[175]:

```
#6
#Зададим массив значений
data = [0.5, 0.698, 0.458, 0.645, 0.2204]

#Затем сгенерируем массив ошибок (отклонений от исходных значений)
n = 10
y = [mean(d*randn(n)) for d in data]
errs = 1.96 * data / sqrt(n)

plot(y, ylims = (-1,1), err = errs)
```

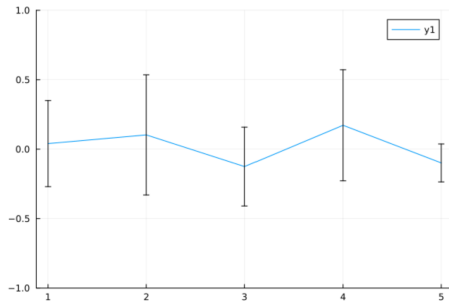


Рис. 60: Задание для самостоятельного выполнения №6

Задания для самостоятельного выполнения

7. Постройте точечный график случайных данных. Подпишите оси, легенду, название графика.

[176]:

```
#7  
x = rand(100)  
y = rand(100)  
scatter(x, y, xlabel="X", ylabel="Y", label="XY", title="Random Data")
```

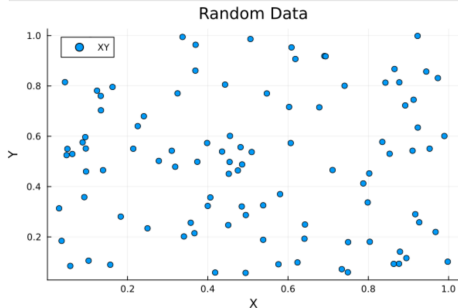


Рис. 61: Задание для самостоятельного выполнения №7

Задания для самостоятельного выполнения

8. Постройте 3-мерный точечный график случайных данных. Подпишите оси, легенду, название графика.

[177]:

```
#8  
( )  
x = rand(100)  
y = rand(100)  
z = rand(100)  
scatter(x, y, z, xlabel="X", ylabel="Y", zlabel="Z", label="XYZ", title="Random Data")
```

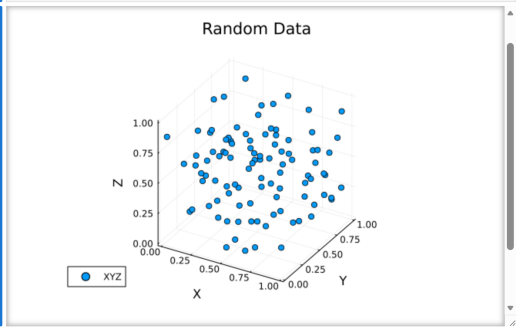


Рис. 62: Задание для самостоятельного выполнения №8

Задания для самостоятельного выполнения

9. Создайте анимацию с построением синусоиды. То есть вы строите последовательность графиков синусоиды, постепенно увеличивая значение аргумента. После соедините их в анимацию.

[178]:

```
#9
pyplot()
x=collect(0:0.1:2*pi)
y7(x)=sin(x)
plot(x, y7.(x))
# анимация:
@gif for i in range(0, stop=10π, length=100)
    x=collect(0:0.1:2*pi+i)
    plot(x, y7.(x))
end
```

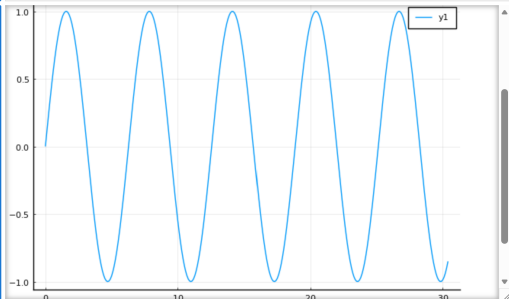


Рис. 63: Задание для самостоятельного выполнения №9

Задания для самостоятельного выполнения

10. Постройте анимированную гипоциклоиду для 2 целых значений модуля k и 2 рациональных значений модуля k .

•[179]:

```
# радиус малой окружности:
r1 = 1
# коэффициент для построения большой окружности:
k = 2
# число отсчётов:
n = 100
# массив значений угла  $\theta$ :
# theta from 0 to 2pi ( + a little extra)
 $\theta$  = collect(0:2* $\pi$ /100:2* $\pi$ +2* $\pi$ /100)
# массивы значений координат:
X = r1*k*cos. $\theta$ 
Y = r1*k*sin. $\theta$ 
anim = @animate for i in 1:n
# задаём оси координат:
    plt=plot(5,xlim=(-4,4),ylim=(-4,4), c=:red, aspect_ratio=1,legend=false, framestyle=:or
# большая окружность:
    plot!(plt, X,Y, c=:blue, legend=false)
    t =  $\theta$ [1:i]
# гипоциклоида:
    x = r1*(k-1)*cos.(t) + r1*cos.((k-1)*t)
    y = r1*(k-1)*sin.(t) - r1*sin.((k-1)*t)
    plot!(x,y, c=:red)
# малая окружность:
    xc = r1*(k-1)*cos(t[end]) .+ r1*cos. $\theta$ 
    yc = r1*(k-1)*sin(t[end]) .+ r1*sin. $\theta$ 
    plot!(xc,yc,c=:black)
# радиус малой окружности:
    x1 = transpose([r1*(k-1)*cos(t[end]) x[end]])
    y1 = transpose([r1*(k-1)*sin(t[end]) y[end]])
    plot!(x1,y1,markershape=:circle,markersize=4,c=:black)
    scatter!([x[end]], [y[end]], c=:red, markerstrokecolor=:red)
end
gif(anim, "hypocycloid1.gif")
```


Задания для самостоятельного выполнения

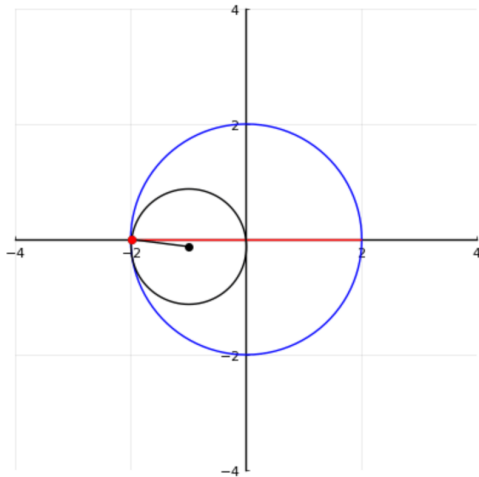


Рис. 65: Задание для самостоятельного выполнения №10

Задания для самостоятельного выполнения

```
# радиус малой окружности:
r1 = 1
# коэффициент для построения большой окружности:
k = 6
# число отсчётов:
n = 100
# массив значений угла  $\theta$ :
# theta from  $\theta$  to  $2\pi$  ( + a little extra)
 $\theta = \text{collect}(\theta:2*\pi/100:2*\pi+2*\pi/100)$ 
# массивы значений координат:
X = r1*k*cos.( $\theta$ )
Y = r1*k*sin.( $\theta$ )
anim = @animate for i in 1:n
# задаём оси координат:
    plt=plot(5,xlim=(-10,10),ylim=(-10,10), c=:red, aspect_ratio=1,legend=false, framestyle=:none)
# большая окружность:
    plot!(plt, X,Y, c=:blue, legend=false)
    t =  $\theta[1:i]$ 
# гипоциклоида:
    x = r1*(k-1)*cos.(t) + r1*cos.((k-1)*t)
    y = r1*(k-1)*sin.(t) - r1*sin.((k-1)*t)
    plot!(x,y, c=:red)
# малая окружность:
    xc = r1*(k-1)*cos(t[end]) .+ r1*cos.( $\theta$ )
    yc = r1*(k-1)*sin(t[end]) .+ r1*sin.( $\theta$ )
    plot!(xc,yc,c=:black)
# радиус малой окружности:
    x1 = transpose([r1*(k-1)*cos(t[end]) x[end]])
    y1 = transpose([r1*(k-1)*sin(t[end]) y[end]])
    plot!(x1,y1,markershape=:circle,markersize=4,c=:black)
    scatter!([x[end],[y[end]],c=:red, markerstrokecolor=:red)
end
gif(anim,"hypocycloid2.gif")
```

Рис. 66: Задание для самостоятельного выполнения №10

Задания для самостоятельного выполнения

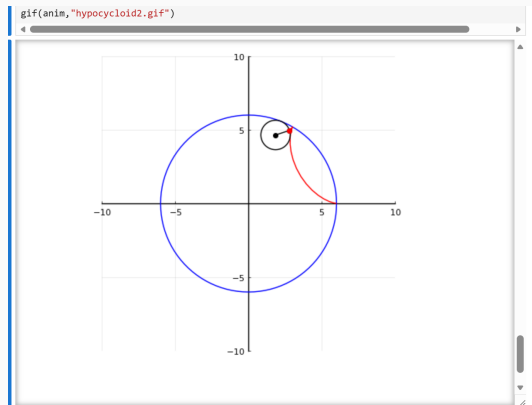


Рис. 67: Задание для самостоятельного выполнения №10

Задания для самостоятельного выполнения

```
# радиус малой окружности:
r1 = 1
# коэффициент для построения большой окружности:
k = 22//3
# число отсчётов:
n = 100
# массив значений угла θ:
# theta from 0 to 2pi ( + a little extra)
θ = collect(0:2*π/100:2*π+2*π/100)
# массивы значений координат:
X = r1*k*cos.(θ)
Y = r1*k*sin.(θ)
anim = @animate for i in 1:n
# задаём оси координат:
    plt=plot(5,xlim=(-10,10),ylim=(-10,10), c=:red, aspect_ratio=1,legend=false, framestyle=:none)
# большая окружность:
    plot!(plt, X,Y, c=:blue, legend=false)
    t = θ[1:i]
# гипоциклоида:
    x = r1*(k-1)*cos.(t) + r1*cos.((k-1)*t)
    y = r1*(k-1)*sin.(t) - r1*sin.((k-1)*t)
    plot!(x,y, c=:red)
# малая окружность:
    xc = r1*(k-1)*cos(t[end]) + r1*cos.(θ)
    yc = r1*(k-1)*sin(t[end]) + r1*sin.(θ)
    plot!(xc,yc,c=:black)
# радиус малой окружности:
    x1 = transpose([r1*(k-1)*cos(t[end]) x[end]])
    y1 = transpose([r1*(k-1)*sin(t[end]) y[end]])
    plot!(x1,y1,markershape=:circle,markersize=4,c=:black)
    scatter!([x[end]], [y[end]], c=:red, markerstrokecolor=:red)
end
gif(anim, "hypocycloid3.gif")
```

Рис. 68: Задание для самостоятельного выполнения №10

Задания для самостоятельного выполнения

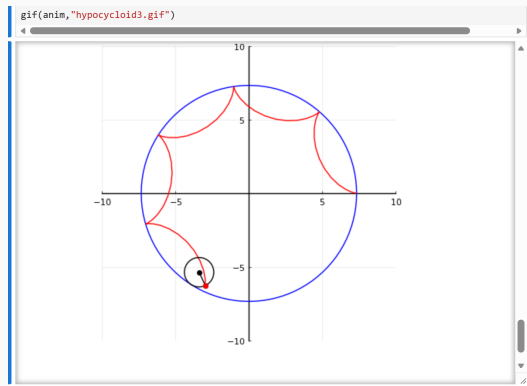


Рис. 69: Задание для самостоятельного выполнения №10

Задания для самостоятельного выполнения

```
# радиус малой окружности:
r1 = 1
# коэффициент для построения большой окружности:
k = 10//7
# число отсчётов:
n = 100
# массив значений угла  $\theta$ :
# theta from  $\theta$  to  $2\pi$  ( + a little extra)
 $\theta$  = collect( $\theta$ :2* $\pi$ /100:2* $\pi$ +2* $\pi$ /100)
# массивы значений координат:
X = r1*k*cos.( $\theta$ )
Y = r1*k*sin.( $\theta$ )

anim = @animate for i in 1:n
# задаём оси координат:
    plt=plot(5,xlim=(-4,4),ylim=(-4,4), c=:red, aspect_ratio=1,legend=false, framestyle=:ori
# большая окружность:
    plot!(plt, X,Y, c=:blue, legend=false)
    t =  $\theta$ [1:i]
# гипоциклоида:
    x = r1*(k-1)*cos.(t) + r1*cos.((k-1)*t)
    y = r1*(k-1)*sin.(t) - r1*sin.((k-1)*t)
    plot!(x,y, c=:red)
# малая окружность:
    xc = r1*(k-1)*cos(t[end]) .+ r1*cos.( $\theta$ )
    yc = r1*(k-1)*sin(t[end]) .+ r1*sin.( $\theta$ )
    plot!(xc,yc,c=:black)
# радиус малой окружности:
    x1 = transpose([r1*(k-1)*cos(t[end]) x[end]])
    y1 = transpose([r1*(k-1)*sin(t[end]) y[end]])
    plot!(x1,y1,markershape=:circle,markersize=4,c=:black)
    scatter!([x[end]], [y[end]], c=:red, markerstrokecolor=:red)
end
gif(anim, "hypocycloid4.gif")
```

Рис. 70: Задание для самостоятельного выполнения №10

Задания для самостоятельного выполнения

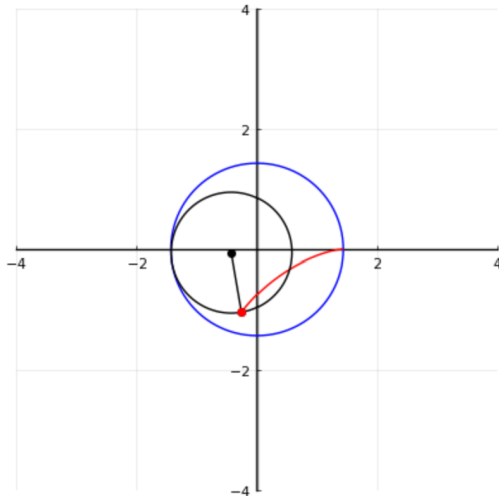


Рис. 71: Задание для самостоятельного выполнения №10

Задания для самостоятельного выполнения

11. Постройте анимированную эпициклоиду для 2 целых значений модуля k и 2 рациональных значений модуля k .

```
•[181]:  
  
# радиус малой окружности:  
r1 = 1  
# коэффициент для построения большой окружности:  
k = 2  
# число отсчётов:  
n = 100  
# массив значений угла  $\theta$ :  
# theta from 0 to 2pi ( + a little extra)  
 $\theta$  = collect(0:2*pi/100:2*pi+2*pi/100)  
# массивы значений координат:  
X = r1*k*cos. $\theta$   
Y = r1*k*sin. $\theta$   
anim = @animate for i in 1:n  
# задаём оси координат:  
    plt=plot(5,xlim=(-4,4),ylim=(-4,4), c=:red, aspect_ratio=1,legend=false, framestyle=:or  
# большая окружность:  
    plot!(plt, X,Y, c=:blue, legend=false)  
    t =  $\theta$ [1:i]  
# эпициклоида:  
    x = r1*(k+1)*cos.(t) - r1*cos.((k+1)*t)  
    y = r1*(k+1)*sin.(t) - r1*sin.((k+1)*t)  
    plot!(x,y, c=:red)  
# малая окружность:  
    xc = r1*(k+1)*cos(t[end]) .- r1*cos. $\theta$   
    yc = r1*(k+1)*sin(t[end]) .- r1*sin. $\theta$   
    plot!(xc,yc,c=:black)  
# радиус малой окружности:  
    x1 = transpose([r1*(k+1)*cos(t[end]) x[end]])  
    y1 = transpose([r1*(k+1)*sin(t[end]) y[end]])  
    plot!(x1,y1,markershape=:circle,markersize=4,c=:black)  
    scatter!([x[end]],y[end],c=:red, markerstrokecolor=:red)  
end  
gif(anim, "epicycloid1.gif")
```


Задания для самостоятельного выполнения

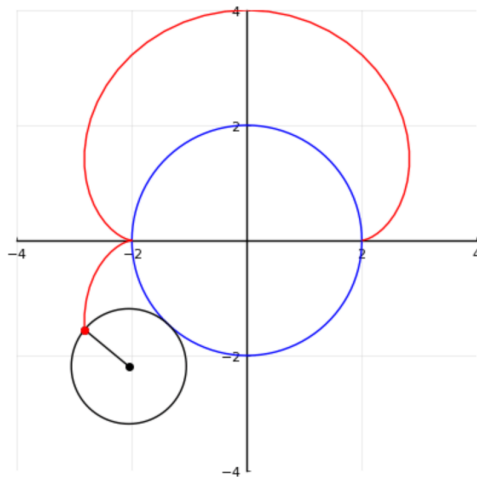


Рис. 73: Задание для самостоятельного выполнения №11

Задания для самостоятельного выполнения

```
# радиус малой окружности:
r1 = 1
# коэффициент для построения большой окружности:
k = 6
# число отсчётов:
n = 100
# массив значений угла  $\theta$ :
# theta from  $\theta$  to  $2\pi$  ( + a little extra)
 $\theta = \text{collect}(\theta:2*\pi/100:2*\pi+2*\pi/100)$ 
# массивы значений координат:
X = r1*k*cos.( $\theta$ )
Y = r1*k*sin.( $\theta$ )
anim = @animate for i in 1:n
# задаём оси координат:
    plt=plt(5,xlim=(-10,10),ylim=(-10,10), c=:red, aspect_ratio=1,legend=false, framestyle=
# большая окружность:
    plot!(plt, X,Y, c=:blue, legend=false)
    t =  $\theta[1:i]$ 
# эпициклоида:
    x = r1*(k+1)*cos.(t) - r1*cos.((k+1)*t)
    y = r1*(k+1)*sin.(t) - r1*sin.((k+1)*t)
    plot!(x,y, c=:red)
# малая окружность:
    xc = r1*(k+1)*cos(t[end]) - r1*cos.( $\theta$ )
    yc = r1*(k+1)*sin(t[end]) - r1*sin.( $\theta$ )
    plot!(xc,yc,c=:black)
# радиус малой окружности:
    x1 = transpose([r1*(k+1)*cos(t[end]) x[end]])
    y1 = transpose([r1*(k+1)*sin(t[end]) y[end]])
    plot!(x1,y1,markershape=:circle,markersize=4,c=:black)
    scatter!([x[end]],[y[end]],c=:red, markerstrokecolor=:red)
end
gif(anim,"epicycloid2.gif")
```

Рис. 74: Задание для самостоятельного выполнения №11

Задания для самостоятельного выполнения

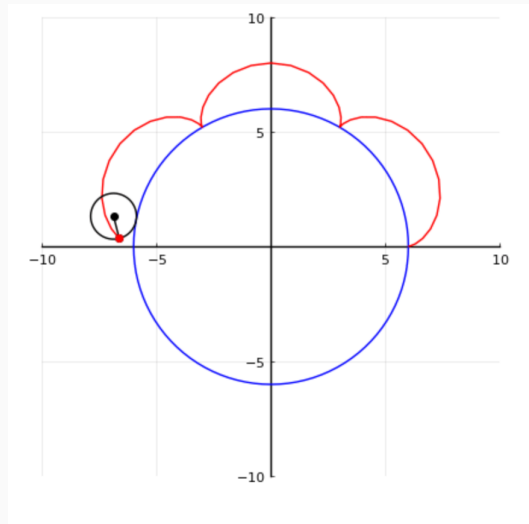


Рис. 75: Задание для самостоятельного выполнения №11

Задания для самостоятельного выполнения

```
# радиус малой окружности:
r1 = 1
# коэффициент для построения большой окружности:
k = 22//3
# число отсчётов:
n = 100
# массив значений угла θ:
# theta from 0 to 2pi ( + a little extra)
θ = collect(0:2*π/100:2*π+2*π/100)
# массивы значений координат:
X = r1*k*cos.(θ)
Y = r1*k*sin.(θ)
anim = @animate for i in 1:n
# задаём оси координат:
    plt=plot(5,xlim=(-10,10),ylim=(-10,10), c=:red, aspect_ratio=1,legend=false, framestyle=:none)
# большая окружность:
    plot!(plt, X,Y, c=:blue, legend=false)
    t = θ[1:i]
# эписиклоида:
    x = r1*(k+1)*cos.(t) - r1*cos.((k-1)*t)
    y = r1*(k+1)*sin.(t) - r1*sin.((k-1)*t)
    plot!(x,y, c=:red)
# малая окружность:
    xc = r1*(k+1)*cos(t[end]) .- r1*cos.(θ)
    yc = r1*(k+1)*sin(t[end]) .- r1*sin.(θ)
    plot!(xc,yc,c=:black)
# радиус малой окружности:
    x1 = transpose([r1*(k+1)*cos(t[end]) x[end]])
    y1 = transpose([r1*(k+1)*sin(t[end]) y[end]])
    plot!(x1,y1,markershape=:circle,markersize=4,c=:black)
    scatter!([x[end]], [y[end]], c=:red, markerstrokecolor=:red)
end
gif(anim, "epicycloid3.gif")
```

Рис. 76: Задание для самостоятельного выполнения №11

Задания для самостоятельного выполнения

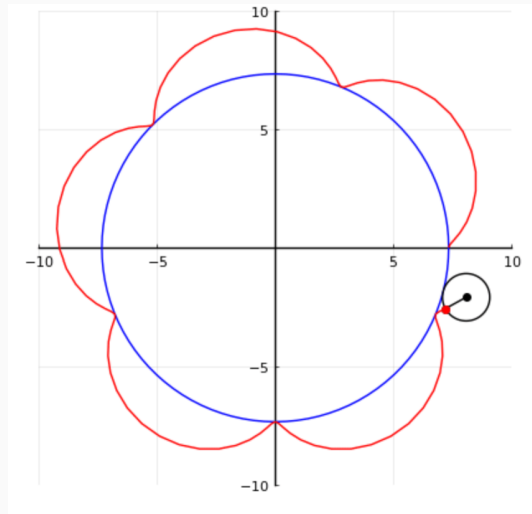


Рис. 77: Задание для самостоятельного выполнения №11

Задания для самостоятельного выполнения

```
# радиус малой окружности:
r1 = 1
# коэффициент для построения большой окружности:
k = 10//7
# число отсчётов:
n = 100
# массив значений угла  $\theta$ :
# theta from 0 to 2pi ( + a little extra)
 $\theta$  = collect(0:2*pi/100:2*pi+2*pi/100)
# массивы значений координат:
X = r1*k*cos.( $\theta$ )
Y = r1*k*sin.( $\theta$ )

anim = @animate for i in 1:n
# задаём оси координат:
plt=plot(5,xlim=(-4,4),ylim=(-4,4), c=:red, aspect_ratio=1,legend=false, framestyle=:orig)
# большая окружность:
plot!(plt, X,Y, c=:blue, legend=false)
t =  $\theta[1:i]$ 
# эписклоида:
x = r1*(k+1)*cos.(t) - r1*cos.((k-1)*t)
y = r1*(k+1)*sin.(t) - r1*sin.((k-1)*t)
plot!(x,y, c=:red)
# малая окружность:
xc = r1*(k+1)*cos(t[end]) .- r1*cos.( $\theta$ )
yc = r1*(k+1)*sin(t[end]) .- r1*sin.( $\theta$ )
plot!(xc,yc,c=:black)
# радиус малой окружности:
x1 = transpose([r1*(k+1)*cos(t[end]) x[end]])
y1 = transpose([r1*(k+1)*sin(t[end]) y[end]])
plot!(x1,y1,markershape=:circle,markersize=4,c=:black)
scatter!([x[end]],y[end]),c=:red, markerstrokecolor=:red)
end
gif(anim,"epicycloid4.gif")
```

Рис. 78: Задание для самостоятельного выполнения №11

Задания для самостоятельного выполнения

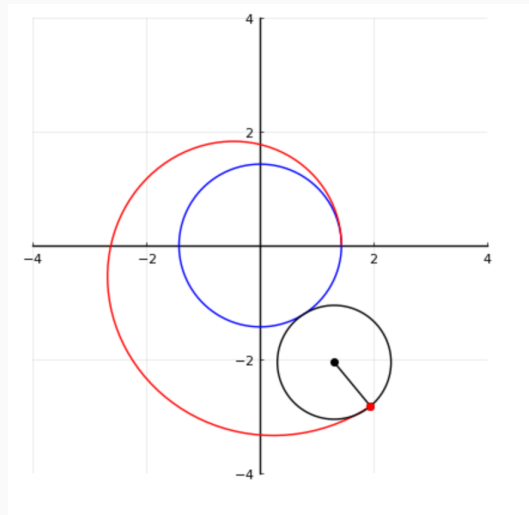


Рис. 79: Задание для самостоятельного выполнения №11

В результате выполнения работы был освоен синтаксис языка Julia для построения графиков.